

如何將 Ollama 託管為推論工作站集區

來源：<https://codelabs.developers.google.com/codelabs/cloud-run/host-ollama-worker-pool?hl=zh-tw>

步驟數：8

瞭解如何將 Ollama 設為推論的 worker 集區

目錄

1. [簡介](#)
2. [事前準備](#)
3. [設定和需求](#)
4. [下載模型並在 GCS 上代管](#)
5. [建立 Cloud Run 工作](#)
6. [部署及執行工作](#)
7. [恭喜！](#)
8. [清除所用資源](#)

步驟 1 · 約 0 分鐘

1. 簡介

總覽

在本程式碼研究室中，您將瞭解如何建構事件驅動的非同步 AI 處理管道。您將在 Cloud Run Worker 集區上，使用 Ollama 部署開放原始碼模型。工作站集區會從 Pub/Sub 主題提取訊息，並使用 gemma3:4b 模型處理這些訊息。

課程內容

- 如何搭配 Pub/Sub 提取訂閱項目使用工作人員集區
 - 如何使用 Ollama 做為工作站集區進行推論
-

步驟 2 · 約 0 分鐘

2. 事前準備

啟用 API

開始使用本程式碼研究室前，請先執行下列指令，啟用下列 API：

```
gcloud services enable run.googleapis.com \  
  cloudbuild.googleapis.com \  
  artifactregistry.googleapis.com \  
  pubsub.googleapis.com \  
  storage.googleapis.com
```

3. 設定和需求

如要設定必要資源，請按照下列步驟操作：

1. 為本程式碼研究室設定環境變數：



```
export PROJECT_ID=<YOUR_PROJECT_ID>
export REGION=<YOUR_REGION>

export BUCKET_NAME=$PROJECT_ID-gemma3-4b
export SERVICE_ACCOUNT_NAME=ollama-worker-sa
export SERVICE_ACCOUNT_EMAIL=${SERVICE_ACCOUNT_NAME}@${PROJECT_ID}.iam.gserviceaccount.com
export TOPIC_NAME=ollama-prompts
export SUBSCRIPTION_NAME=ollama-prompts-sub
export AR_REPO_NAME=ollama-worker-repo
export PULL_MSG_IMAGE_NAME=pubsub-pull-msg
export OLLAMA_IMAGE_NAME=ollama-coordinator
```

2. 為工作站集區建立服務帳戶



```
gcloud iam service-accounts create ${SERVICE_ACCOUNT_NAME} \
  --display-name="Ollama Worker Service Account"
```

3. 授予服務帳戶 Pub/Sub 存取權



```
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${SERVICE_ACCOUNT_EMAIL}" \
  --role="roles/pubsub.subscriber"
```

4. 為工作站集區映像檔建立 AR 存放區

```
gcloud artifacts repositories create ${AR_REPO_NAME} \
  --repository-format=docker \
  --location=${REGION}
```

5. 建立 Pub/Sub 主題和訂閱項目

```
gcloud pubsub topics create $TOPIC_NAME
gcloud pubsub subscriptions create $SUBSCRIPTION_NAME --topic $TOPIC_NAME
```

步驟 4 · 約 0 分鐘

4. 下載模型並在 GCS 上代管

我們不會在建構程序期間直接在容器內提取模型 (這樣可能很慢且效率不彰)，而是使用 Ollama CLI 將模型提取至本機，然後將模型檔案上傳至 GCS bucket。worker 集區隨後會掛接這個 bucket，以存取模型。

1. 在本機安裝 Ollama：

在 Linux 上執行下列指令，安裝 Ollama。如需其他作業系統的資訊，請參閱 [Ollama 網站](#)。

```
curl -fsSL https://ollama.com/install.sh | sh
```

2. 啟動 Ollama 服務並提取模型：

首先，在背景啟動 Ollama 服務。

```
ollama serve &  
ollama pull gemma3:4b
```

4. 建立 GCS bucket：

使用您先前設定的 `BUCKET_NAME` 環境變數建立 GCS bucket。

```
gsutil mb gs://${BUCKET_NAME}
```

4. 將模型檔案上傳至 GCS bucket：

Ollama 會將模型檔案儲存在 `~/.ollama/models` 目錄中。將這個目錄的內容上傳至 GCS bucket。系統會複製所有已下載的模型。

```
gsutil -m cp -r ~/.ollama/models/* gs://${BUCKET_NAME}/
```

5. 授予服務帳戶 Cloud Storage bucket 的存取權

```
gcloud storage buckets add-iam-policy-binding gs://${BUCKET_NAME} \  
  --member=serviceAccount:${SERVICE_ACCOUNT_EMAIL} \  
  --role=roles/storage.objectViewer
```

步驟 5 · 約 0 分鐘

5. 建立 Cloud Run 工作

Cloud Run 工作會使用 2 個容器：

- ollama-coordinator：用於託管 ollama 並提供 gemma 3 4B 模型
- pubsub-pull-msg：從 Pub/Sub 訂閱項目提取訊息，並將訊息傳遞至 ollama-coordinator 容器

首先，請建立 ollama-coordinator 容器。

1. 為 Codelab 建立上層目錄：

```
mkdir codelab-ollama-wp
cd codelab-ollama-wp
```

2. 為 ollama-coordinator 容器建立目錄

```
mkdir ollama-coordinator
cd ollama-coordinator
```

3. 建立含有以下內容的 `Dockerfile`

```
# Use the official Ollama image as a base image
FROM ollama/ollama

# Expose the port that Ollama listens on
EXPOSE 11434

# Set the entrypoint to start the Ollama server
ENTRYPOINT ["ollama", "serve"]
```

4. 建構 ollama 容器

```
gcloud builds submit --tag ${REGION}-
docker.pkg.dev/${PROJECT_ID}/${AR_REPO_NAME}/${OLLAMA_IMAGE_NAME} --timeout=20m
```

接下來，您要建立 pubsub-pull-msg 容器。

1. 為 pubsub-pull-msg 容器建立目錄

```
cd ..
mkdir pubsub-pull-msg
cd pubsub-pull-msg
```

2. 建立 `Dockerfile`

```
# Use the official Python image as a base image
FROM python:3.9-slim

# Set the working directory in the container
WORKDIR /app

# Copy the requirements file into the container
COPY requirements.txt .

# Install the required Python packages
RUN pip install --no-cache-dir -r requirements.txt

# Copy the Python script into the container
COPY main.py .

# Set the entrypoint to run the Python script
CMD ["python", "main.py"]
```

3. 建立 `requirements.txt` 檔案，並加入以下內容：

```
google-cloud-pubsub
requests
```

4. 建立 `main.py` 檔案，並加入以下內容：


```

import os
import sys
import requests
import json
from google.cloud import pubsub_v1

# --- Main Application Logic ---
print("--- Sidecar container script started ---")

# --- Environment and Configuration ---
project_id = os.environ.get("PROJECT_ID")
subscription_name = os.environ.get("SUBSCRIPTION_NAME")
ollama_api_url = "http://localhost:11434/api/generate"

if not project_id or not subscription_name:
    print("FATAL: PROJECT_ID and SUBSCRIPTION_NAME must be set.")
    sys.exit(1)

print(f"PROJECT_ID: {project_id}")
print(f"SUBSCRIPTION_NAME: {subscription_name}")

def callback(message):
    """Processes a single Pub/Sub message."""
    print(f"Received message ID: {message.message_id}")
    try:
        prompt = message.data.decode("utf-8")
        print(f"Decoded prompt: '{prompt}'")

        data = {"model": "gemma3:4b", "prompt": prompt, "stream": False}

        print("Sending request to Ollama...")
        response = requests.post(ollama_api_url, json=data, timeout=300)
        response.raise_for_status()

        print("Successfully received response from Ollama.")
        ollama_response = response.json()
        print(f'Ollama response: {json.dumps(ollama_response)[:200]}...')

        message.ack()
        print(f'Message {message.message_id} acknowledged.')

    except requests.exceptions.RequestException as e:
        print(f'Error calling Ollama API: {e}')
        message.nack()
        print(f'Message {message.message_id} not acknowledged.')
    except Exception as e:
        print(f'An unexpected error occurred in callback: {e}')
        message.nack()
        print(f'Message {message.message_id} not acknowledged.')

def main():
    """Starts the Pub/Sub subscriber."""

```

```
subscriber = pubsub_v1.SubscriberClient()
subscription_path = subscriber.subscription_path(project_id, subscription_name)

streaming_pull_future = subscriber.subscribe(subscription_path, callback=callback)
print(f"Subscribed to {subscription_path}. Listening for messages...")

try:
    # .result() will block indefinitely.
    streaming_pull_future.result()
except Exception as e:
    print(f"A fatal error occurred in the subscriber: {e}")
    streaming_pull_future.cancel()
    streaming_pull_future.result()

if __name__ == "__main__":
    main()
```

5. 現在請建構 pubsub-pull-msg 容器

```
gcloud builds submit --tag ${REGION}-
docker.pkg.dev/${PROJECT_ID}/${AR_REPO_NAME}/${PULL_MSG_IMAGE_NAME}
```

步驟 6 · 約 0 分鐘

6. 部署及執行工作

在這個步驟中，您將部署 YAML 檔案，建立 Cloud Run 工作。

移至根資料夾，建立 YAML 檔案。

```
cd ..
```

1. 建立 `worker-pool.template.yaml` 檔案，並加入以下內容：

```

apiVersion: run.googleapis.com/v1
kind: WorkerPool
metadata:
  name: codelab-ollama-wp
  labels:
    cloud.googleapis.com/location: europe-west1
  annotations:
    run.googleapis.com/launch-stage: BETA
    run.googleapis.com/scalingMode: manual
    run.googleapis.com/manualInstanceCount: '1'
    run.googleapis.com/gcs-fuse-mounter-enabled: "true"
spec:
  template:
    metadata:
      annotations:
        run.googleapis.com/gpu: "1"
        run.googleapis.com/gpu-zonal-redundancy-disabled: 'true'
    spec:
      serviceAccountName: ${SERVICE_ACCOUNT_EMAIL}
      nodeSelector:
        run.googleapis.com/accelerator: nvidia-l4
      volumes:
        - name: gcs-bucket
          csi:
            driver: gcsfuse.run.googleapis.com
            readOnly: true
            volumeAttributes:
              bucketName: ${BUCKET_NAME}
      containers:
        - image: ${REGION}-docker.pkg.dev/${PROJECT_ID}/${AR_REPO_NAME}/${PULL_MSG_IMAGE_NAME}
          name: pubsub-pull-msg
          env:
            - name: PROJECT_ID
              value: ${PROJECT_ID}
            - name: SUBSCRIPTION_NAME
              value: "ollama-prompts-sub"
            - name: PYTHONUNBUFFERED
              value: "1"
          resources:
            limits:
              cpu: '1'
              memory: 1Gi
        - image: ${REGION}-docker.pkg.dev/${PROJECT_ID}/${AR_REPO_NAME}/${OLLAMA_IMAGE_NAME}
          name: ollama-coordinator
          env:
            - name: OLLAMA_MODELS
              value: /mnt/models
          volumeMounts:
            - name: gcs-bucket
              mountPath: /mnt/models
          resources:
            limits:

```

```
cpu: '6'
nvidia.com/gpu: '1'
memory: 16Gi
```

接著，定義完整圖片網址，並使用 `sed` 替換範本檔案中的變數，建立最終的 `worker-pool.yaml`。

```
sed -e "s|\${SERVICE_ACCOUNT_EMAIL}|\${SERVICE_ACCOUNT_EMAIL}|g" \
-e "s|\${BUCKET_NAME}|\${BUCKET_NAME}|g" \
-e "s|\${PULL_MSG_IMAGE_NAME}|\${PULL_MSG_IMAGE_NAME}|g" \
-e "s|\${OLLAMA_IMAGE_NAME}|\${OLLAMA_IMAGE_NAME}|g" \
-e "s|\${PROJECT_ID}|\${PROJECT_ID}|g" \
-e "s|\${REGION}|\${REGION}|g" \
-e "s|\${AR_REPO_NAME}|\${AR_REPO_NAME}|g" \
worker-pool.template.yaml > worker-pool.yaml
```

現在可以部署

```
gcloud beta run worker-pools replace worker-pool.yaml
```

And Test

```
gcloud pubsub topics publish ${TOPIC_NAME} --message="What is 1 + 1?"
```

然後查看記錄。您可能需要等待一分鐘，或前往 [Cloud 控制台工作站集區頁面](#)，即時監看記錄。

```
gcloud alpha run worker-pools logs read "codelab-ollama-wp" --limit 10
```

您應該會看到類似「

```
Ollama response: {"model": "gemma3:4b", "created_at": "2025-11-06T23:48:39.572079369Z",
"response": "1 + 1 = 2\n", ...
```

步驟 7 · 約 0 分鐘

7. 恭喜！

恭喜您完成本程式碼研究室！

建議參閱 [Cloud Run](#) 說明文件。

涵蓋內容

- 如何搭配使用 Cloud Run worker 集區與 Pub/Sub 提取式訂閱項目
 - 如何使用 Ollama 做為 Cloud Run worker 集區執行推論
-

步驟 8 · 約 0 分鐘

8. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本教學課程所用資源的費用，請刪除含有相關資源的專案，或者保留專案但刪除個別資源。

刪除專案

如要避免付費，最簡單的方法就是刪除您為了本教學課程所建立的專案。

如要刪除專案，請進行以下操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關閉) 即可刪除專案。

刪除個別資源

如要刪除個別資源，請執行下列指令：

1. 刪除 Cloud Run worker 集區：

```
gcloud beta run worker-pools delete codelab-ollama-wp --region ${REGION}
```

2. 刪除 GCS bucket：

```
gsutil -m rm -r gs://${BUCKET_NAME}
```

3. 刪除 Pub/Sub 訂閱項目和主題：

```
gcloud pubsub subscriptions delete ${SUBSCRIPTION_NAME}
gcloud pubsub topics delete ${TOPIC_NAME}
```

4. 刪除 Artifact Registry 存放區：

```
gcloud artifacts repositories delete ${AR_REPO_NAME} --location=${REGION} --quiet
```

6. 刪除服務帳戶：

```
gcloud iam service-accounts delete ${SERVICE_ACCOUNT_EMAIL} --quiet
```

清理本機檔案

如要清除本機檔案，請按照下列步驟操作：

1. 停止本機 Ollama 服務：如果您使用 `ollama serve &` 啟動 Ollama，可以找出其程序 ID (PID)，然後使用 `kill` 指令停止服務。

```
# Find the process ID of the Ollama server
```

```
pgrep ollama
```

```
# Replace <PID> with the actual process ID obtained from the previous command
```

```
kill <PID>
```

2. 刪除已下載的模型：

```
rm -rf ~/.ollama/models
```

3. 解除安裝 Ollama：

按照 [Ollama 網站](#)上的說明，從本機解除安裝 Ollama。
